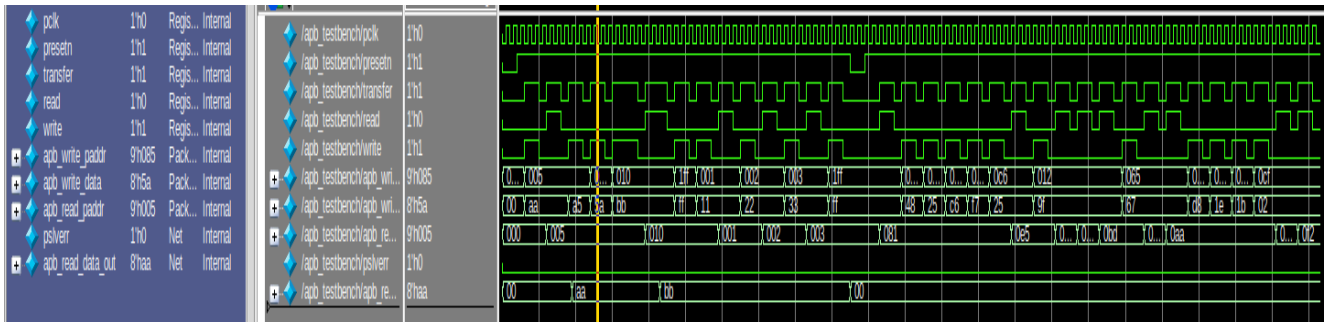
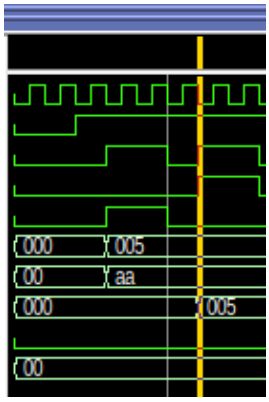
INDUSTRIAL PROTOCOLS LAB 1

NAME: Ali Irfan
ROLL NO: 22FA-036-CE

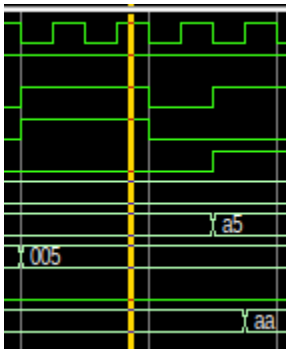
TASK 1:



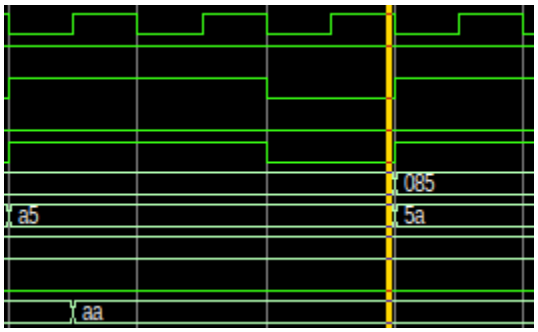
TC1 (Basic Write Operation): The master initiates a write transaction by asserting transfer = 1 and write = 1 with target address 9'h005 and data 8'hAA. Slave responds by asserting pready after 1 clock cycle, and data is successfully written.



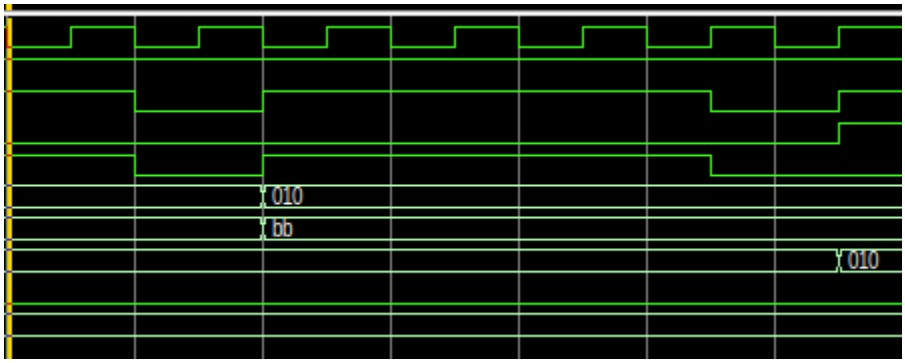
TC2 (Basic Read Operation): The master sets transfer = 1 and read = 1 for address 9'h005. The slave asserts pready to acknowledge and places the requested read data on the data bus.



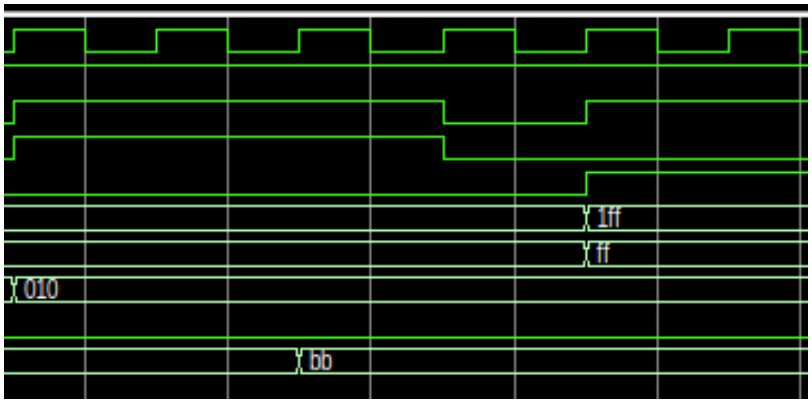
TC3 (Address Decoding): Tests slave selection mechanism based on the MSB bit of the address. Address 9'h005 selects Slave 1 (psel1 = 1), while address 9'h085 selects Slave 2 (psel2 = 1), keeping the inactive slave idle.



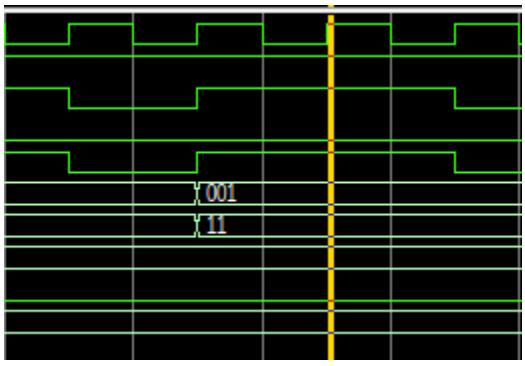
TC4 (Write with Wait States): Simulates a write operation to address 9'h010 where the slave delays asserting pready. The master remains in the ENABLE state and holds data steady until pready goes HIGH.



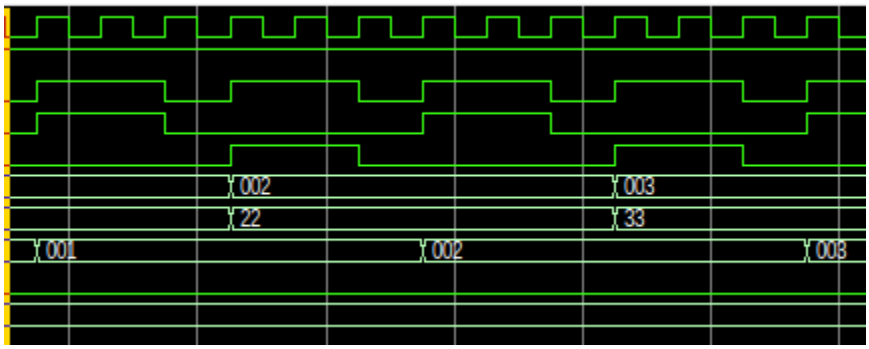
TC5 (Read with Wait States): Simulates a read operation with wait states at address 9'h010. The master stalls until pready is driven HIGH by the slave, successfully capturing the data upon transfer completion.



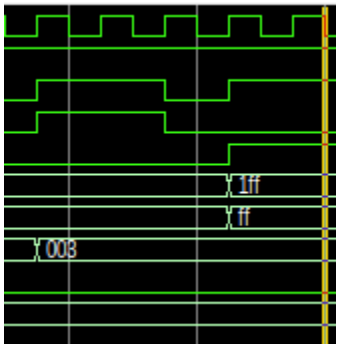
TC6 (Error Handling): An invalid address 9'h1FF is sent to trigger an error condition. The slave asserts pslverr along with pready to notify the master of an out-of-range or illegal transaction.



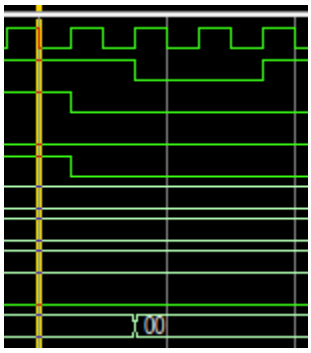
TC7 (Burst Transfers): Executes back-to-back sequential write and read transactions on addresses 9'h001, 9'h002, and 9'h003 without reverting to the IDLE state between transfers.



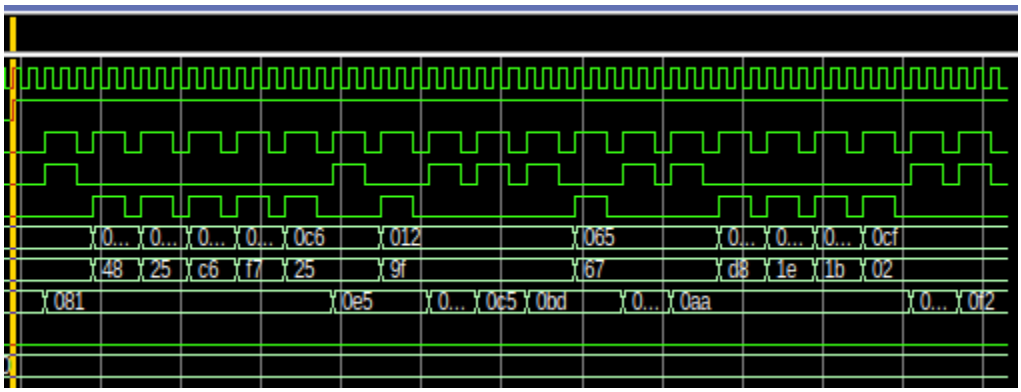
TC8 (Out-of-Range Address): Evaluates system stability when attempting to write to an unmapped/invalid memory location. The slave flag pslverr is checked to ensure illegal access detection.



TC9 (Reset Behavior): Asserts active-low presetn for 20 ns during system operation. All state registers, control signals (penable, psel), and outputs immediately revert to their default reset states.



TC10 (Randomized Transactions): Performs 20 randomized back-to-back read/write operations with varying addresses and data payloads to stress-test protocol stability and signal compliance.



TASK 02:

Design Code:

/home/cl4/Protocols verilog/handshaking.v - Default

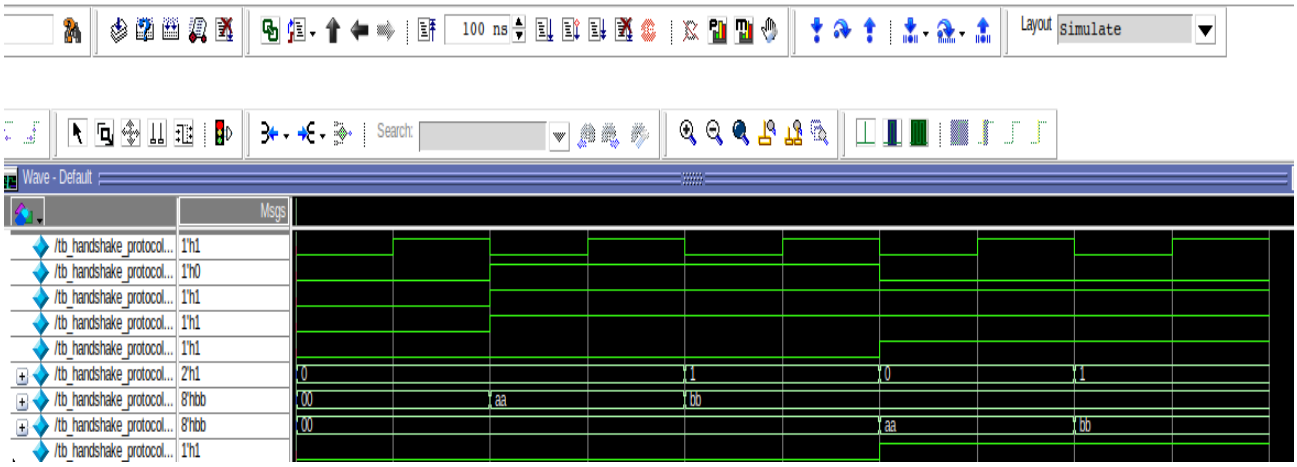
```
 2  input clk,
 3  input read_write,
 4  input valid_m,
 5  input ready_s,
 6  input ready_m,
 7  input [1:0] sel,
 8  input [7:0] data_in,
 9  output reg [7:0] data_out,
10  output valid_s
11  );
12
13  reg [7:0] reg0;
14  reg [7:0] reg1;
15  reg [7:0] reg2;
16  reg [7:0] reg3;
17
18  assign valid_s = (~read_write) & ready_m;
19
20  always@(posedge clk)
21  begin
22  if(read_write & valid_m & ready_s)
23  begin
24  if(sel==2'b00) reg0 <= data_in;
25  if(sel==2'b01) reg1 <= data_in;
26  if(sel==2'b10) reg2 <= data_in;
27  if(sel==2'b11) reg3 <= data_in;
28  end
29  end
30
31  always@(*)
32  begin
33  if((~read_write) & ready_m)
34  begin
35  case(sel)
36  2'b00: data_out = reg0;
37  2'b01: data_out = reg1;
38  2'b10: data_out = reg2;
39  2'b11: data_out = reg3;
40  default: data_out = 8'h00;
41  endcase
42  end
43  else
44  data_out = 8'h00;
45  end
46
47  endmodule
```

Testbench Code:

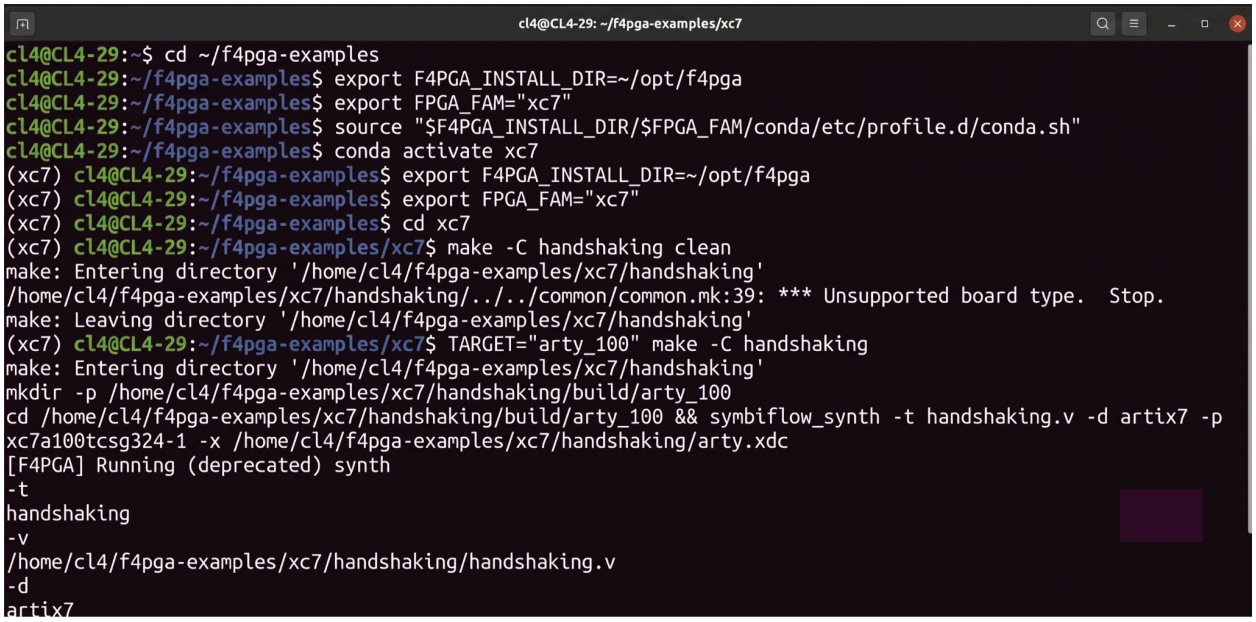
/home/cl4/Protocols verilog/tb_handshaking.v - Default

```
Ln#
 1  module tb_handshake_protocol;
 2  reg clk;
 3  reg read_write;
 4  reg valid_m;
 5  reg ready_s;
 6  reg ready_m;
 7  reg [1:0] sel;
 8  reg [7:0] data_in;
 9  wire [7:0] data_out;
10  wire valid_s;
11
12  handshake_protocol uut(
13  .clk(clk),
14  .read_write(read_write),
15  .valid_m(valid_m),
16  .ready_s(ready_s),
17  .ready_m(ready_m),
18  .sel(sel),
19  .data_in(data_in),
20  .data_out(data_out),
21  .valid_s(valid_s)
22  );
23
24  always #5 clk = ~clk;
25
26  initial begin
27  clk = 0;
28  read_write = 0;
29  valid_m = 0;
30  ready_s = 0;
31  ready_m = 0;
32  sel = 2'b00;
33  data_in = 8'h00;
34  #10;
35  read_write = 1;
36  valid_m = 1;
37  ready_s = 1;
38  sel = 2'b00;
39  data_in = 8'hAA;
40  #10;
41  sel = 2'b01;
42  data_in = 8'hBB;
43  #10;
44  read_write = 0;
45  ready_m = 1;
46  sel = 2'b00;
47  #10;
48  sel = 2'b01;
49  #10;
50  $finish;
51  end
52
53  endmodule
54
```

Waveform:



FPGA Commands:



Description:

- Design Summary: The handshake module uses register-mapped storage (reg0 to reg3) with clear distinction between write (read_write = 1) and read (read_write = 0) operations controlled by valid_m, ready_s, and ready_m signals.
- Write Transfer Condition: Data from data_in is written to the selected register (sel) on a rising clock edge only when both valid_m (master valid) and ready_s (slave ready) are HIGH simultaneously.
- Read Transfer Condition: Output data is driven on data_out and valid_s is asserted only when read_write is LOW and ready_m (master ready) is HIGH.
- FPGA Hardware Observation: When deployed on FPGA, input control signals (valid_m, ready_s, ready_m, read_write) are mapped to board slide switches, and data_out is mapped to output LEDs. If valid_m is asserted without ready_s, no write operation occurs (data holds steady). Only upon asserting ready_s does the transfer complete on the clock edge, updating the internal register.

TASK 03: